

for garbage collection. Garbage Collection is done automatically to return RAM memory back to the system—so called “termination housekeeping.”

We say that an unused object is marked for garbage collection. A finalizer method must always be called `finalize()`. It always takes no arguments and returns `void`. The `finalize()` method is one of the 11 methods inherited from method `Object`.

4.11 Static class methods

When we instantiate a class, each instantiation of that class gets its own private copies of the instance variables for that class. However, in certain cases, we would like to have all the instances of the class share one copy of a variable, instead of each having their own copy.

```
interestRate
```

Say we had 30,000 instances of a class called `SavingsAccount`. If we changed the `interestRate` variable they all contained, we would have to make 30,000 method calls to make the change in all of them. However, if we just had defined the interest variable as static, we would have only a single copy that all of them shared.

```
static interestRate
```

Only the members of the class could access the static variable. Instance variables that are defined as being static have class scope. If you define your static instance variables as `public`:

```
public int static interestRate
```

then this variable can be reached by a reference to any object of that class, or through the class name using the dot operator:

```
savacct.interestRate
```

or

```
SavingsAccount.interestRate
```

Or

```
ClassName.staticFieldName;
```

If you define your static instance variables as `private`:

```
private int static interestRate
```

Then the private static instance variables can only be accessed through methods of the class, like any other private variable. Static Class members can be accessed even when no instances of that class exist. To access a public static class member when the class is not instantiated, you tack the name of the class to the name of the variable:

```
SavingsAccount.interestRate
```